

RapidGo: **Migración Serverless**

Análisis e Implementación de Arquitectura Cloud

EQUIPO DE TRABAJO

Diego Alejandro Barrios

Andrés Camilo Díaz

Jose Alejandro Colorado

1. Caso de Estudio

Desafíos Críticos del Monolito Actual

El Problema de Negocio



Escalabilidad

El monolito colapsa en horas pico (4.500 pedidos). La lentitud en la API genera una tasa de cancelación del 12%, afectando los ingresos directos.



Costos Fijos

Se paga un servidor dedicado de \$4.2M COP al mes. Durante las madrugadas, la infraestructura se desperdicia con solo un 4% de uso de CPU.



Notificaciones

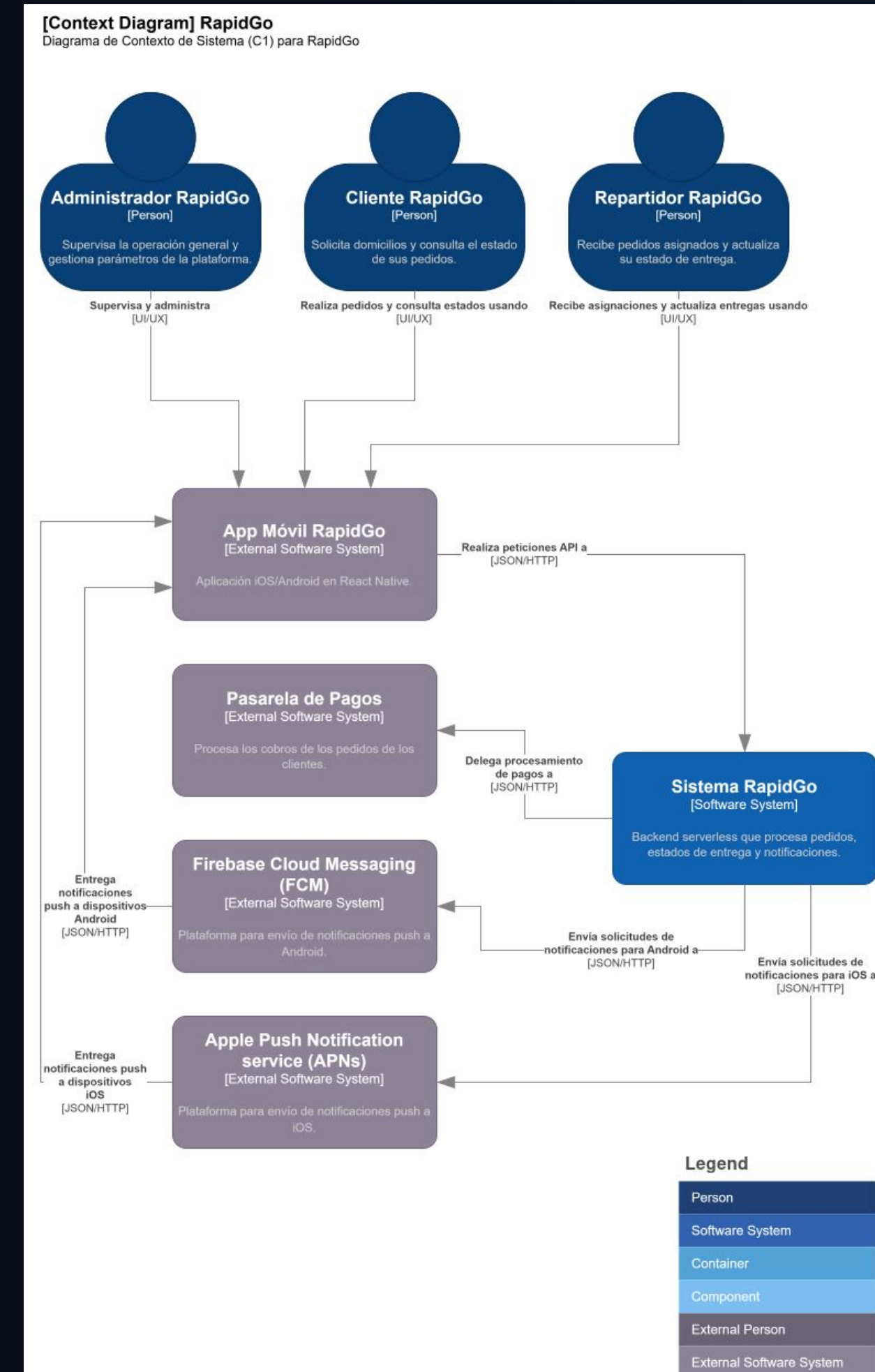
El sistema legado carece de integración directa, logrando apenas una tasa de entrega del 67% en mensajes push, lo que genera confusión en clientes.

2. Arquitectura

Modelo C4: Contexto, Contenedores y Componentes

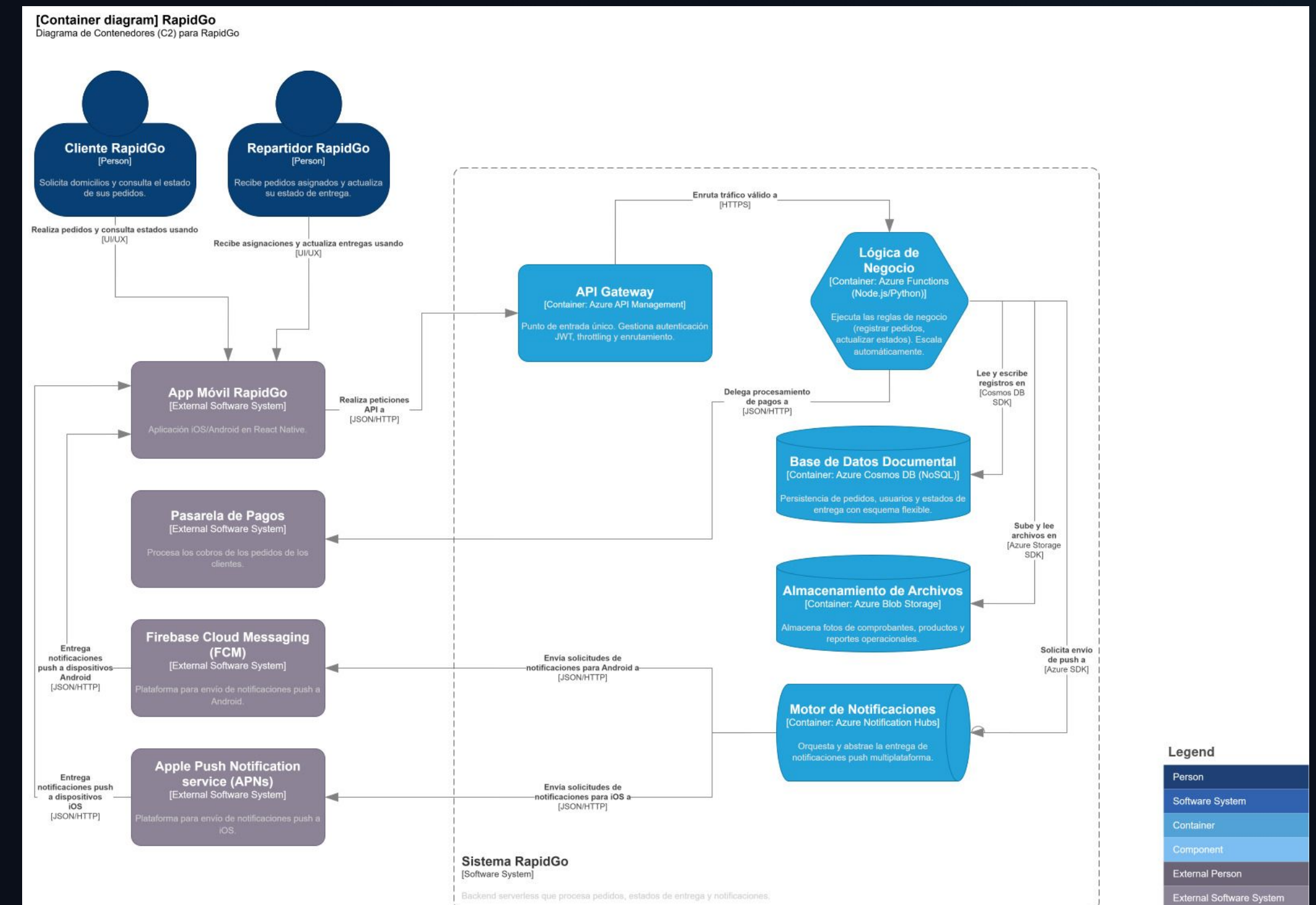
Contexto del Sistema (C1)

- **Visión de alto nivel:** El backend se aísla como una caja negra (Sistema Central).
- **Actores:** Clientes (crean pedidos), Repartidores (actualizan estados) y Administradores (supervisión).
- **Interfaz:** App Móvil construida en React Native.
- **Sistemas Externos:** Pasarela de pagos para cobros y proveedores FCM/APNs para notificaciones push.



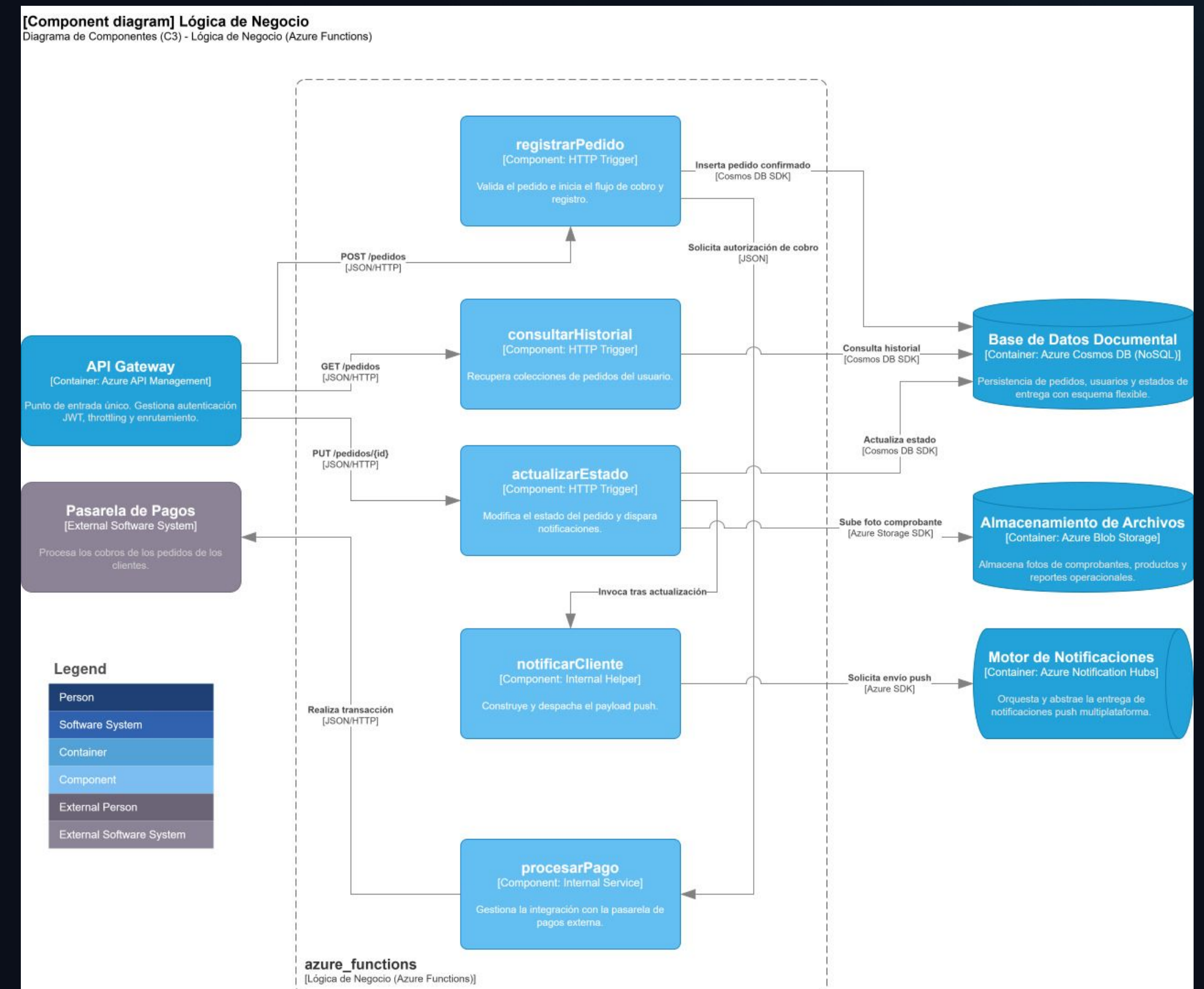
Ecosistema Cloud (C2)

- **API Management:** Actúa como puerta de entrada. Valida seguridad (JWT) y protege el backend.
- **Azure Functions:** El núcleo stateless del negocio. Escala de 0 a N instancias automáticamente.
- **Cosmos DB:** Base de datos NoSQL ultra-rápida. Su esquema JSON absorbe atributos variables.
- **Blob Storage:** Almacena masivamente imágenes y comprobantes de entrega a muy bajo costo.
- **Notification Hubs:** Motor unificado para garantizar entregas push a iOS y Android.



Componentes Internos (C3)

- **registrarPedido:** Recibe POST, solicita cobro y guarda en Cosmos DB.
- **consultarHistorial:** Resuelve peticiones GET desde Cosmos DB.
- **actualizarEstado:** Recibe PUT, actualiza la BD y sube evidencias al Blob.
- **procesarPago:** Servicio interno que aísla la comunicación con la pasarela.
- **notificarCliente:** Helper que delega el envío push a Notification Hubs.



3. Justificación

Decisiones Arquitectónicas Documentadas (ADRs)

Motores Principales

ADR-01: Azure Functions

Se descartó el App Service (servidor permanente) en favor del **Consumption Plan** (Serverless). Esta decisión elimina el gasto fijo: solo se paga por milisegundos de ejecución. La infraestructura ahora absorbe los picos de forma autónoma.

ADR-02: Cosmos DB (NoSQL)

A pesar del histórico en MySQL, se justificó un cambio de paradigma hacia una base de datos **documental**. Esto resuelve el requerimiento del negocio de guardar atributos variables sin romper esquemas rígidos.

Soporte e Integración



ADR-03: API Mgt

Descarga la seguridad del código base y adapta los contratos para mantener intacto el Frontend actual en React Native.



ADR-04: Blobs

Se eligió Blob Storage frente a Azure Files por ser la solución óptima y de menor costo para objetos no estructurados.



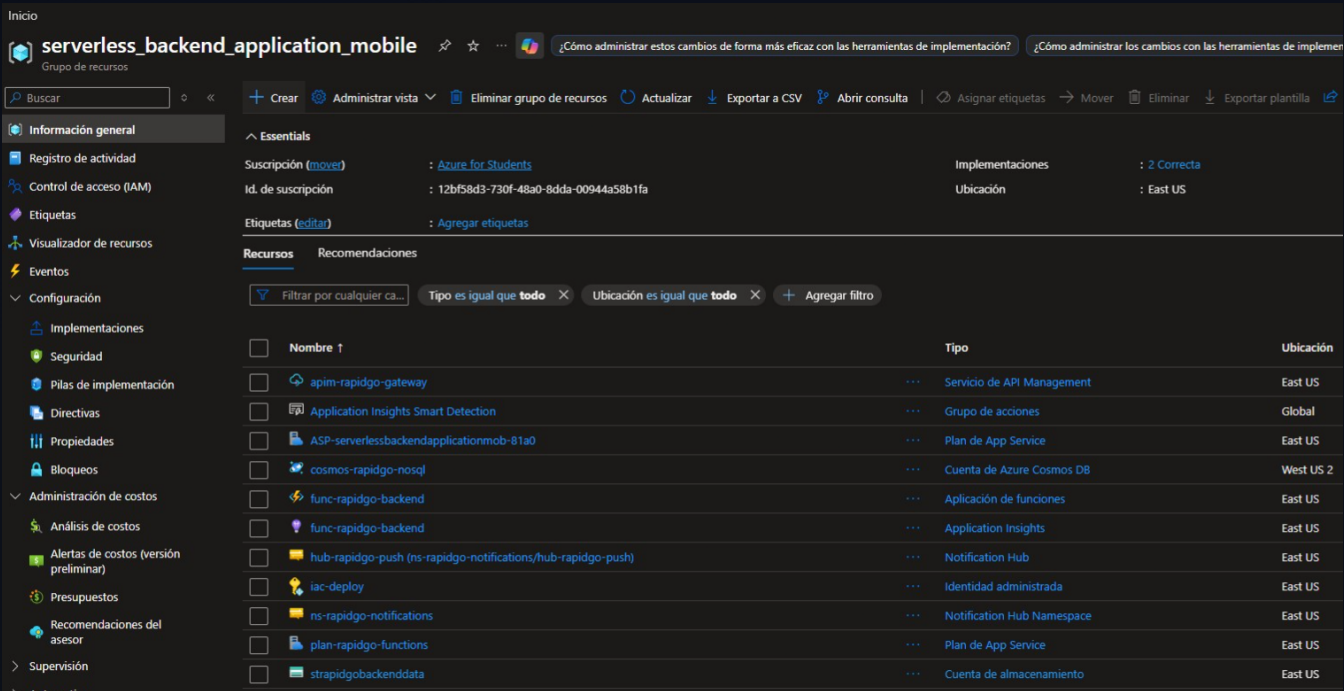
ADR-05: Hubs

Abstrae la complejidad de hablar con Apple y Google por separado, asegurando el SLA requerido (>95% de entrega).

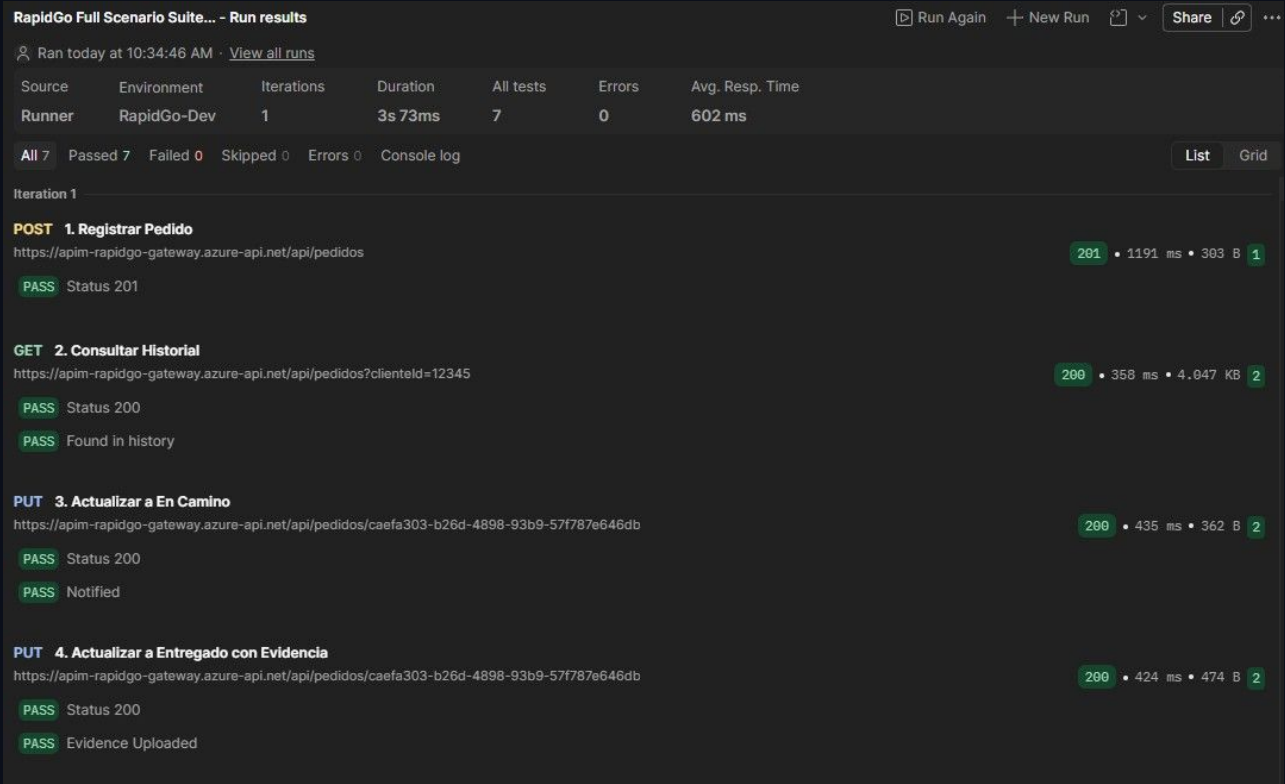
4. Funcionamiento

Evidencias del Flujo Crítico

Implementación Exitosa



Infraestructura: Recursos Cloud
aprovisionados exitosamente en Azure.



Flujo Crítico: Pruebas E2E de creación y
actualización de estados.

✓ Merge pull request #28 from AlejoBarriosDev/fix/notification-metrics-... #94

Summary

All jobs

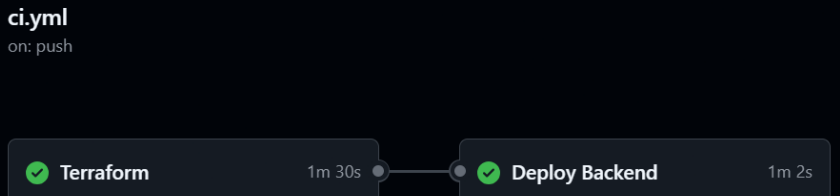
- ✓ Terraform
- ✓ Deploy Backend

Run details

Usage

Workflow file

Triggered via push 20 hours ago
AlejoBarriosDev pushed -o- 4729bb7 main
Status: Success
Total duration: 2m 38s



Automatización: Logs de despliegues y
automatización de infraestructura.

5. Conclusiones

Aprendizajes y Retos Técnicos

Conclusiones

Retos Serverless

Superamos restricciones de zonas en Cosmos DB (migrando a West US 2) y resolvimos bloqueos del plan Flex Consumption usando empaquetados ZIP.

Secretos Seguros

Para no sobrecomplejizar la arquitectura con Key Vault, inyectamos las credenciales de Apple/Google usando variables `sensitive=true` en Terraform.

Notificaciones

Ante incompatibilidades del SDK con Python 3.11, desarrollamos una solución robusta consumiendo directamente la API REST del Notification Hub.

CI/CD y Módulos

Evitamos el "código espagueti" con Blueprints, y garantizamos cero caídas orquestando despliegues automáticos con GitHub Actions.

Lógica vs Física

El diagrama C2 muestra componentes de negocio, pero el portal Azure revela recursos técnicos extra (ej. Storage Account) obligatorios para la Function App.

Pruebas y Costos

Para evitar gastos en Azure durante el desarrollo, implementamos `unittest.mock` localmente, dejando Postman solo para validaciones finales E2E.

¿Preguntas?

Gracias por su atención